

Lab_Assignment_6.2

AI Assisted Coding B37 2303A52077

Task -1 (Classes – Data Validation)

- Prompt AI to generate a student class with attributes: name, roll_no, and marks. Add a method is_pass() that returns whether the student has passed (marks ≥ 40)

Prompt:

#write a python code to to generate a student class with attributes: name, roll_no, and marks,

Add a method is_pass() that returns whether the student has passed (marks ≥ 40).

Code & Output:

```
sixth.py  X
sixth.py > ...
1  #write a python code to to generate a student class with attributes: name, roll_no, and marks,
2  # Add a method is_pass() that returns whether the student has passed (marks  $\geq 40$ ).
3  class Student:
4      def __init__(self, name, roll_no, marks):
5          self.name = name
6          self.roll_no = roll_no
7          self.marks = marks
8
9      def is_pass(self):
10         return self.marks >= 40
11
12  # Example usage:
13  student1 = Student("Alice", 1, 45)
14  print(f"Student: {student1.name}, Roll No: {student1.roll_no}, Marks: {student1.marks}, Passed: {student1.is_pass()}")
15  student2 = Student("Bob", 2, 35)
16  print(f"Student: {student2.name}, Roll No: {student2.roll_no}, Marks: {student2.marks}, Passed: {student2.is_pass()}")
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
PS D:\Subjects\ai assisted coding\ai_assi> & C:/Users/h.py"
Student: Alice, Roll No: 1, Marks: 45, Passed: True
Student: Bob, Roll No: 2, Marks: 35, Passed: False
PS D:\Subjects\ai assisted coding\ai_assi>
```

Explanation:

1. In this code, we define a Student class with an initializer method (`__init__`) that sets the attributes `name`, `roll_no`, and `marks` (Parameterized Constructor).
2. The `is_pass()` method checks if the marks are greater than or equal to 40 and returns `True` if the student has passed, otherwise it returns `False`.

Task -2 (Loops – Pattern Generation)

- Ask AI to generate a function that prints a right-angled triangle star pattern using a for loop. Then regenerate the same pattern using a while loop.

Prompt(For_Loop):

- #write a python for loop to print right angle triangle
- # pattern with stars pattern with given number of rows

Code & Output(For_Loop):

```
17 #write a python for loop to print right angle triangle
18 # pattern with stars pattern with given number of rows
19 def print_triangle_pattern(rows):
20     for i in range(1, rows + 1):
21         print('*' * i)
22 # Example usage:
23 print_triangle_pattern(5)
```

```
*
**
***
****
*****
```

Explanation(print_triangle_pattern function):

- The function `print_triangle_pattern` takes an integer input 'rows' which specifies the number of rows for the triangle pattern.
- It uses a for loop that iterates from 1 to 'rows' (inclusive).
- In each iteration, it prints a string of asterisks (*) multiplied by the current row number 'i'.
- This results in the first row having 1 star, the second row having 2 stars, and so on, up to the specified number of rows.
- The output is a right-angled triangle pattern made of stars.

Prompt(While_Loop):

#write a python while loop to print right angle triangle

pattern with stars pattern with given number of rows

Code & Output:

```
24 #write a python while loop to print right angle triangle
25 # pattern with stars pattern with given number of rows
26 def print_triangle_pattern_while(rows):
27     i = 1
28     while i <= rows:
29         print('*' * i)
30         i += 1
31 # Example usage:
32 print_triangle_pattern_while(5)
```

```
*
**
***
****
*****
```

Explanation (print_triangle_pattern_while):

- The function `print_triangle_pattern_while` takes an integer parameter 'rows' which specifies the number of rows for the triangle pattern.

- It initializes a variable 'i' to 1, which will be used to keep track of the current row number.
- The while loop continues as long as 'i' is less than or equal to 'rows'.
- Inside the loop, it prints a string of asterisks (*) repeated 'i' times, which creates the right angle triangle pattern.
- After printing the stars for the current row, it increments 'i' by 1 to move to the next row.

Observation:

Both the for loop and while loop implementations achieve the same result of printing a right angle triangle pattern with stars. However, the for loop is generally considered better in this case for several reasons:

1. Readability: The for loop is more concise and easier to read, as it clearly indicates the range of iteration in a single line.

2. Less Error-Prone: The for loop automatically handles the initialization, condition checking, and incrementing of the loop variable, reducing the chances of errors such as infinite loops or incorrect increments that can occur with while loops.

3. Performance: While the performance difference is negligible for small loops, for loops can be slightly more efficient in terms of execution time in some cases, as they are optimized for iteration.

Overall, for loops are often preferred for iterating over a known range or collection, while loops are better suited for situations where the number of iterations is not predetermined. is better suited for situations where the number of iterations is not predetermined. are better suited for situations where the number of iterations is not predetermined.

In conclusion, for loops are generally more suitable for this specific use case of printing a triangle pattern. are better suited for situations where the number of iterations is not predetermined

Task -3 (Conditional Statements – Number Analysis)

- Ask AI to write a function that checks whether a given number is positive, negative, or zero using if-elif-else. Test the function with multiple inputs.

Prompt:

#write a python function that checks whether a given
number is positive, negative, or zero using if-elif-else

Code & Output:

```
34  #write a python function that checks whether a given
35  # number is positive, negative, or zero using if-elif-else
36  def check_number(num):
37      if num > 0:
38          return "Positive"
39      elif num < 0:
40          return "Negative"
41      else:
42          return "Zero"
43  # Example usage:
44  print(check_number(10))    # Output: Positive
45  print(check_number(-5))   # Output: Negative
46  print(check_number(0))    # Output: Zero
```

```
Positive
Negative
Zero
PS D:\Subjects\ai assisted coding\ai_assi> █
```

Explanation:

The function `check\_number` takes a single argument `num`, which is expected to be a numeric value. The purpose of this function is to determine whether the given number is positive, negative, or zero.

Here's a breakdown of how the function works:

1. **Function Definition:** The function is defined using the ``def`` keyword, followed by the function name ``check_number`` and a parameter ``num``.
2. **If Statement:** The first condition checks if ``num`` is greater than 0. If this condition is true, the function returns the string "Positive".
3. **Elif Statement:** If the first condition is false, the function checks the second condition using ``elif``. It checks if ``num`` is less than 0. If this condition is true, the function returns the string "Negative".
4. **Else Statement:** If neither of the previous conditions is true (meaning ``num`` is neither greater than nor less than 0), the function executes the ``else`` block, returning the string "Zero".
5. **Return Values:** The function returns one of three strings based on the value of ``num``: "Positive", "Negative", or "Zero".

Task Description-4 (Nested Conditionals)

- Generate a function `check_discount(age, is_member)` that determines discount eligibility:
- $\text{Age} \geq 60 \rightarrow \text{Senior discount}$
- $\text{Member} \rightarrow \text{Additional discount}$

Use nested if statements.

Prompt:

'''

write a python function `check_discount(age, is_member)` that determines discount eligibility:

$\text{age} \geq 60 \rightarrow \text{Senior discount}$

member → Additional discount

also use nested if statements.

'''

Code & Output:

```
48 '''
49 write a python function check_discount(age, is_member) that
50 determines discount eligibility:
51 age ≥ 60 → Senior discount
52 member → Additional discount
53 also use nested if statements.
54 '''
55 def check_discount(age, is_member):
56     if age >= 60:
57         discount = "Senior discount"
58         if is_member:
59             discount += " with Additional member discount"
60     else:
61         if is_member:
62             discount = "Member discount"
63         else:
64             discount = "No discount"
65     return discount
66 # Example usage:
67 print(check_discount(65, True)) # Output: Senior discount with Additional member discount
68 print(check_discount(30, True)) # Output: Member discount
69 print(check_discount(70, False)) # Output: Senior discount
70
```

```
Senior discount with Additional member discount
Member discount
Senior discount
PS D:\Subjects\ai assisted coding\ai_assi> █
```

Explanation:

The function `check\_discount(age, is\_member)` determines the type of discount a person is eligible for based on their age and membership status.

1. It first checks if the person's age is 60 or older. If true, it assigns "Senior discount" to the `discount` variable.

2. Inside this block, it checks if the person is a member ('is_member' is True). If they are, it appends " with Additional member discount" to the 'discount' variable.
3. If the person's age is less than 60, it checks if they are a member. If they are, it assigns "Member discount" to the 'discount' variable.
4. If the person is neither a senior nor a member, it assigns "No discount" to the 'discount' variable.
5. Finally, the function returns the 'discount' variable indicating the type of discount the person is eligible for.

Task -5 (Class – Mathematical Opera)

- Ask AI to create a Circle class with methods to calculate area () and circumference () given the radius.

Prompt: #write a python Circle class with methods to calculate area () and circumference () given the radius

Code & Output:

```
70
71 #write a python Circle class with methods to calculate area () and circumference () given the radius
72 import math
73 class Circle:
74     def __init__(self, radius):
75         self.radius = radius
76
77     def area(self):
78         return math.pi * (self.radius ** 2)
79
80     def circumference(self):
81         return 2 * math.pi * self.radius
82
83 # Example usage:
84 circle = Circle(5)
85 print(f"Area: {circle.area()}") # Output: Area: 78.53981633974483
86 print(f"Circumference: {circle.circumference()}") # Output: Circumference: 31.41592653589793
87
```

Area: 78.53981633974483
Circumference: 31.41592653589793

Explanation

The Circle class is designed to represent a circle and provide methods to calculate its area and circumference based on its radius.

Here's a breakdown of the Circle class:

1. Class Definition:

- The class is defined using the keyword `class` followed by the class name `Circle`

2. `__init__` Method:

- The `__init__` method is a special method in Python classes, known as the constructor.

- It is called when an instance (object) of the class is created.

- It takes `self` (the instance being created) and `radius` (the radius of the circle) as parameters.

- Inside the `__init__` method, the `radius` parameter is assigned to an instance variable `self.radius`,

- which allows it to be accessed by other methods within the class.

3. `area()` Method:

- The `area()` method calculates the area of the circle using the formula πr^2

- It uses the `math.pi` constant to get the value of π .

- The method returns the calculated area.

4. `circumference()` Method:

- The `circumference()` method calculates the circumference of the circle using the formula $2\pi r$.

- Similar to the `area()` method, it uses `math.pi` for the value of π .

- The method returns the calculated circumference.

Conclusion:

1. Student Class:

- A class named Student is defined with an `__init__` method that initializes the attributes

 - name, roll_no, and marks.

- The `is_pass()` method checks if the student's marks are 40 or above,

 - returning True if the student has passed and False otherwise.

2. Triangle Pattern with For Loop:

- The function `print_triangle_pattern(rows)` uses a for loop to print a right-angle triangle

 - pattern made of stars. It iterates from 1 to the specified number of rows,

 - printing a line of stars corresponding to the current row number.

3. Triangle Pattern with While Loop:

- The function `print_triangle_pattern_while(rows)` achieves the same result as the previous function

 - but uses a while loop instead. It initializes a counter and increments it until it reaches the number of rows.

4. Number Check Function:

- The function `check_number(num)` checks whether a given number is positive, negative,

 - or zero using if-elif-else statements and returns the corresponding string.

5. Discount Eligibility Function:

- The function `check_discount(age, is_member)` determines discount eligibility based on age and

membership status. It uses nested if statements to check if the age is 60 or above for a senior discount

and whether the person is a member for an additional discount.

6. Circle Class:

- A class named Circle is defined with an `__init__` method that initializes the radius attribute.

- The `area()` method calculates the area of the circle using the formula πr^2 .

- The `circumference()` method calculates the circumference using the formula $2\pi r$.

Each part of the code includes example usage to demonstrate how the classes and functions work.